

DL_Assignment_2_Group_201

August 14, 2025

0.1 Group No: 201

0.2 Group Member Names:

1. RAMESH P.: 2024AB05263
2. JAGADI SHRUTI CHANABASAPPA: 2024AA05181
3. TANAY KASHYAP: 2024AA05991
4. SHIKHAR NIGAM: 2024AA05691

0.3 Journal used for the implementation

Journal title: Hybrid CNN-Transformer Architecture for Pneumonia Detection in Chest X-rays

Authors: Kim, Lee & Park

Journal Name: IEEE Transactions on Medical Imaging

Year: 2024

Dataset URL: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

1 1. Import the required libraries

```
[9]: ##-----Type the code below this line-----##
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, \
    ModelCheckpoint
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.metrics import SparseCategoricalAccuracy

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.metrics import classification_report, confusion_matrix, \
    roc_auc_score, roc_curve
from sklearn.utils.class_weight import compute_class_weight
import os
import cv2
from PIL import Image
import warnings
warnings.filterwarnings('ignore')

# Set random seeds for reproducibility
tf.random.set_seed(42)
np.random.seed(42)

print(f"TensorFlow version: {tf.__version__}")
print(f"Keras version: {keras.__version__}")
print("GPU Available: ", tf.test.is_gpu_available())
print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))

```

```

TensorFlow version: 2.19.0
Keras version: 3.10.0
GPU Available: False
Num GPUs Available: 0

```

2 2. Data Acquisition

Dataset URL: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

This dataset contains chest X-ray images organized into training, validation, and test sets. - **NORMAL:** No pneumonia detected - **PNEUMONIA:** Pneumonia detected (bacterial or viral)

The dataset contains 5,863 X-Ray images (JPEG) and 2 categories (Pneumonia/Normal).

```

[10]: import os
import kagglehub

# Try to find dataset in cache first
try:
    path = kagglehub.dataset_download("paultimothymooney/chest-xray-pneumonia")
    print("Dataset available at:", path)
except Exception as e:
    print("Error downloading dataset:", e)
    raise

# Set dataset paths
BASE_DIR = os.path.join(path, "chest_xray")
TRAIN_DIR = os.path.join(BASE_DIR, 'train')
VAL_DIR = os.path.join(BASE_DIR, 'val')
TEST_DIR = os.path.join(BASE_DIR, 'test')

```

```

# Data parameters
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
CLASSES = ['NORMAL', 'PNEUMONIA']
NUM_CLASSES = len(CLASSES)

# Function to count images in dataset
def count_images(directory):
    counts = {}
    for class_name in CLASSES:
        class_dir = os.path.join(directory, class_name)
        if os.path.exists(class_dir):
            counts[class_name] = len(os.listdir(class_dir))
        else:
            counts[class_name] = 0
    return counts

print("Dataset Distribution:")
print(f"Training: {count_images(TRAIN_DIR)}")
print(f"Validation: {count_images(VAL_DIR)}")
print(f"Testing: {count_images(TEST_DIR)}")

total_train = sum(count_images(TRAIN_DIR).values())
total_val = sum(count_images(VAL_DIR).values())
total_test = sum(count_images(TEST_DIR).values())

print(f"\nTotal Images - Train: {total_train}, Val: {total_val}, Test: {total_test}")

```

Dataset available at:

C:\Users\iamta\.cache\kagglehub\datasets\paultimothymooney\chest-xray-pneumonia\versions\2

Dataset Distribution:

Training: {'NORMAL': 1341, 'PNEUMONIA': 3875}

Validation: {'NORMAL': 8, 'PNEUMONIA': 8}

Testing: {'NORMAL': 234, 'PNEUMONIA': 390}

Total Images - Train: 5216, Val: 16, Test: 624

3. Data Preparation

Comprehensive data preprocessing pipeline including augmentation, normalization, and batch preparation.

```

[11]: # 1. Sanity check: Ensure dataset is not empty
if total_train == 0 or total_val == 0 or total_test == 0:

```

```

        raise ValueError("One of the dataset splits is empty. Please verify dataset_
        integrity.")

# 2. Data augmentation for training
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    zoom_range=0.2,
    shear_range=0.15,
    fill_mode='nearest',
    brightness_range=[0.8, 1.2],
    channel_shift_range=0.1
)

# 3. Validation/Test data generator (no augmentation, only scaling)
val_test_datagen = ImageDataGenerator(rescale=1./255)

# 4. Data generators
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='sparse',
    classes=CLASSES,
    shuffle=True,
    seed=42
)

val_generator = val_test_datagen.flow_from_directory(
    VAL_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='sparse',
    classes=CLASSES,
    shuffle=False
)

test_generator = val_test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=IMG_SIZE,
    batch_size=1,
    class_mode='sparse',
    classes=CLASSES,
    shuffle=False
)

```

```

)

# 5. Print class indices
print(f"\nClass indices mapping: {train_generator.class_indices}")

# 6. Class weights for balanced training
train_class_counts = count_images(TRAIN_DIR)
total_samples = sum(train_class_counts.values())
class_weight = {
    0: total_samples / (2 * train_class_counts['NORMAL']),
    1: total_samples / (2 * train_class_counts['PNEUMONIA'])
}
print(f"Class Weights: {class_weight}")

# 7. Preview first batch shapes
x_batch, y_batch = next(train_generator)
print(f"\nSample batch - Images shape: {x_batch.shape}, Labels shape: {y_batch.
↪shape}")

# 8. Quick sanity check - one sample label distribution from train set
unique, counts = np.unique(y_batch, return_counts=True)
print("Sample batch label distribution:", dict(zip(unique, counts)))

```

Found 5216 images belonging to 2 classes.

Found 16 images belonging to 2 classes.

Found 624 images belonging to 2 classes.

Class indices mapping: {'NORMAL': 0, 'PNEUMONIA': 1}

Class Weights: {0: 1.9448173005219984, 1: 0.6730322580645162}

Sample batch - Images shape: (32, 224, 224, 3), Labels shape: (32,)

Sample batch label distribution: {np.float32(0.0): np.int64(12),
np.float32(1.0): np.int64(20)}

3.1 4. Deep Neural Network Architecture

3.1.1 4.1 Design the architecture that you will be using

3.1.2 4.2 DNN Report

Report the following and provide justification for the same.

- Number of layers
- Number of units in each layer
- Total number of trainable parameters

```

[12]: def hybrid_cnn_transformer(input_shape=(224, 224, 3), num_classes=2,
↪patch_size=16, projection_dim=64, transformer_layers=6):
    inputs = layers.Input(shape=input_shape)

```

```

# --- CNN Feature Extractor (8 layers) ---
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(inputs)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D((2,2))(x)

x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D((2,2))(x)

x = layers.Conv2D(128, (3,3), activation='relu', padding='same')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(128, (3,3), activation='relu', padding='same')(x)
x = layers.BatchNormalization()(x)
cnn_features = layers.MaxPooling2D((2,2))(x) # (H/8, W/8, 128)

# --- Flatten CNN output into patches for Transformer ---
# Convert feature maps into sequence of patches
patch_h = cnn_features.shape[1] // patch_size if cnn_features.shape[1] else 1
patch_w = cnn_features.shape[2] // patch_size if cnn_features.shape[2] else 1

x = layers.Reshape((-1, cnn_features.shape[-1]))(cnn_features)
x = layers.Dense(projection_dim)(x) # Linear projection to transformer dim

# --- Transformer Encoder ---
for _ in range(transformer_layers):
    # Multi-head self-attention
    attn_output = layers.MultiHeadAttention(num_heads=4,
key_dim=projection_dim)(x, x)
    x = layers.Add()([x, attn_output])
    x = layers.LayerNormalization()(x)

    # Feed-forward network
    ffn = layers.Dense(projection_dim*4, activation='relu')(x)
    ffn = layers.Dense(projection_dim)(ffn)
    x = layers.Add()([x, ffn])
    x = layers.LayerNormalization()(x)

# --- Classification Head ---
x = layers.GlobalAveragePooling1D()(x)
x = layers.Dropout(0.3)(x)

```

```

    outputs = layers.Dense(num_classes, activation='softmax')(x)

    model = models.Model(inputs, outputs, name="Hybrid_CNN_Transformer")
    return model

# Instantiate the model
model = hybrid_cnn_transformer(input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3),
    ↪ num_classes=NUM_CLASSES)

# Summary
model.summary()

```

Model: "Hybrid_CNN_Transformer"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0	-
conv2d_6 (Conv2D)	(None, 224, 224, 32)	896	input_layer_1[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 224, 224, 32)	128	conv2d_6[0][0]
conv2d_7 (Conv2D)	(None, 224, 224, 32)	9,248	batch_normalizati...
batch_normalizatio... (BatchNormalizatio...	(None, 224, 224, 32)	128	conv2d_7[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 112, 112, 32)	0	batch_normalizati...
conv2d_8 (Conv2D)	(None, 112, 112, 64)	18,496	max_pooling2d_3[...
batch_normalizatio... (BatchNormalizatio...	(None, 112, 112, 64)	256	conv2d_8[0][0]
conv2d_9 (Conv2D)	(None, 112, 112, 64)	36,928	batch_normalizati...
batch_normalizatio... (BatchNormalizatio...	(None, 112, 112, 64)	256	conv2d_9[0][0]

max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 64)	0	batch_normalizat...
conv2d_10 (Conv2D)	(None, 56, 56, 128)	73,856	max_pooling2d_4[...
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 128)	512	conv2d_10[0][0]
conv2d_11 (Conv2D)	(None, 56, 56, 128)	147,584	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 128)	512	conv2d_11[0][0]
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 128)	0	batch_normalizat...
reshape_1 (Reshape)	(None, 784, 128)	0	max_pooling2d_5[...
dense_14 (Dense)	(None, 784, 64)	8,256	reshape_1[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 784, 64)	66,368	dense_14[0][0], dense_14[0][0]
add_12 (Add)	(None, 784, 64)	0	dense_14[0][0], multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_12[0][0]
dense_15 (Dense)	(None, 784, 256)	16,640	layer_normalizati...
dense_16 (Dense)	(None, 784, 64)	16,448	dense_15[0][0]
add_13 (Add)	(None, 784, 64)	0	layer_normalizati... dense_16[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_13[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 784, 64)	66,368	layer_normalizati... layer_normalizati...
add_14 (Add)	(None, 784, 64)	0	layer_normalizati... multi_head_atten...
layer_normalizatio...	(None, 784, 64)	128	add_14[0][0]

(LayerNormalizatio...			
dense_17 (Dense)	(None, 784, 256)	16,640	layer_normalizat...
dense_18 (Dense)	(None, 784, 64)	16,448	dense_17[0][0]
add_15 (Add)	(None, 784, 64)	0	layer_normalizat... dense_18[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_15[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 784, 64)	66,368	layer_normalizat... layer_normalizat...
add_16 (Add)	(None, 784, 64)	0	layer_normalizat... multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_16[0][0]
dense_19 (Dense)	(None, 784, 256)	16,640	layer_normalizat...
dense_20 (Dense)	(None, 784, 64)	16,448	dense_19[0][0]
add_17 (Add)	(None, 784, 64)	0	layer_normalizat... dense_20[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_17[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 784, 64)	66,368	layer_normalizat... layer_normalizat...
add_18 (Add)	(None, 784, 64)	0	layer_normalizat... multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_18[0][0]
dense_21 (Dense)	(None, 784, 256)	16,640	layer_normalizat...
dense_22 (Dense)	(None, 784, 64)	16,448	dense_21[0][0]
add_19 (Add)	(None, 784, 64)	0	layer_normalizat... dense_22[0][0]
layer_normalizatio...	(None, 784, 64)	128	add_19[0][0]

(LayerNormalizatio...			
multi_head_attentio... (MultiHeadAttentio...	(None, 784, 64)	66,368	layer_normalizat... layer_normalizat...
add_20 (Add)	(None, 784, 64)	0	layer_normalizat... multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_20[0][0]
dense_23 (Dense)	(None, 784, 256)	16,640	layer_normalizat...
dense_24 (Dense)	(None, 784, 64)	16,448	dense_23[0][0]
add_21 (Add)	(None, 784, 64)	0	layer_normalizat... dense_24[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_21[0][0]
multi_head_attentio... (MultiHeadAttentio...	(None, 784, 64)	66,368	layer_normalizat... layer_normalizat...
add_22 (Add)	(None, 784, 64)	0	layer_normalizat... multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_22[0][0]
dense_25 (Dense)	(None, 784, 256)	16,640	layer_normalizat...
dense_26 (Dense)	(None, 784, 64)	16,448	dense_25[0][0]
add_23 (Add)	(None, 784, 64)	0	layer_normalizat... dense_26[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 784, 64)	128	add_23[0][0]
global_average_poo... (GlobalAveragePool...	(None, 64)	0	layer_normalizat...
dropout_13 (Dropout)	(None, 64)	0	global_average_p...
dense_27 (Dense)	(None, 2)	130	dropout_13[0][0]

Total params: 895,458 (3.42 MB)

Trainable params: 894,562 (3.41 MB)

Non-trainable params: 896 (3.50 KB)

```
[ ]: # 5. Model Compilation
model.compile(
    optimizer=Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999),
    loss=SparseCategoricalCrossentropy(from_logits=False),
    metrics=[
        SparseCategoricalAccuracy(name='accuracy')
        # Removed precision and recall due to Keras 3.x compatibility issues
    ]
)

print(" Model compiled successfully!")
print(f"Optimizer: Adam (lr=0.001)")
print(f"Loss: SparseCategoricalCrossentropy")
print(f"Metrics: Accuracy")

# Display model configuration
print(f"\nModel Configuration Summary:")
print(f"- Total Parameters: {model.count_params():,}")
print(f"- Model Name: {model.name}")
print(f"- Input Shape: {model.input_shape}")
print(f"- Output Shape: {model.output_shape}")
```

```
Model compiled successfully!
Optimizer: Adam (lr=0.001)
Loss: SparseCategoricalCrossentropy
Metrics: Accuracy
```

```
Model Configuration Summary:
- Total Parameters: 895,458
- Model Name: Hybrid_CNN_Transformer
- Input Shape: (None, 224, 224, 3)
- Output Shape: (None, 2)
```

4 5. Training the model

```
[24]: from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, \
      ↪ ReduceLROnPlateau
import time
import numpy as np
from sklearn.utils.class_weight import compute_class_weight

# Optional: compute class weights
class_weight_values = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)
class_weight = dict(enumerate(class_weight_values))

# 1. Model checkpoint - save best model
checkpoint_filepath = f'best_hybrid_cnn_transformer_{int(time.time())}.keras'
model_checkpoint = ModelCheckpoint(
    checkpoint_filepath,
    monitor='val_accuracy',
    save_best_only=True,
    save_weights_only=False,
    mode='max',
    verbose=1
)

# 2. Early stopping - prevent overfitting
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True,
    verbose=1,
    mode='min'
)

# 3. Learning rate reduction - adaptive learning
reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,
    min_lr=1e-7,
    verbose=1,
    mode='min'
)

# 4. Combine all callbacks
```

```

callbacks = [
    early_stopping,
    reduce_lr,
    model_checkpoint
]

# 5. Training parameters
EPOCHS = 3
VALIDATION_STEPS = int(np.ceil(val_generator.samples / val_generator.
    ↪batch_size))
STEPS_PER_EPOCH = int(np.ceil(train_generator.samples / train_generator.
    ↪batch_size))

print("Training Configuration:")
print(f" Epochs: {EPOCHS}")
print(f" Steps per epoch: {STEPS_PER_EPOCH}")
print(f" Validation steps: {VALIDATION_STEPS}")
print(f" Model checkpoint: {checkpoint_filepath}")
print(f" Callbacks configured: {len(callbacks)}")
print(f" Class weights: {class_weight}")

print("\n Training callbacks and configuration ready!")

```

```

Training Configuration:
Epochs: 3
Steps per epoch: 163
Validation steps: 1
Model checkpoint: best_hybrid_cnn_transformer_1755109384.keras
Callbacks configured: 3
Class weights: {0: np.float64(1.9448173005219984), 1:
np.float64(0.6730322580645162)}

```

Training callbacks and configuration ready!

```

[25]: # 6. Model Training
# Train the Hybrid CNN-Transformer model with all configured callbacks

import time
from datetime import datetime

# Start training timer
start_time = time.time()

# Train the model
try:
    history = model.fit(
        train_generator,

```

```

        epochs=EPOCHS,
        steps_per_epoch=STEPS_PER_EPOCH,
        validation_data=val_generator,
        validation_steps=VALIDATION_STEPS,
        class_weight=class_weight,
        callbacks=callbacks,
        verbose=1,
        shuffle=True
    )

    # Training completed successfully
    end_time = time.time()
    training_duration = end_time - start_time

    print("\n" + "=" * 60)
    print(" TRAINING COMPLETED SUCCESSFULLY!")
    print("=" * 60)
    print(f"    Total training time: {training_duration//60:.0f} minutes,
↪{training_duration%60:.1f} seconds")
    print(f"    Total epochs completed: {len(history.history['loss'])}")
    print(f"    Best validation accuracy: {max(history.history['val_accuracy']):.
↪4f}")
    print(f"    Final training loss: {history.history['loss'][-1]:.4f}")
    print(f"    Final validation loss: {history.history['val_loss'][-1]:.4f}")

except Exception as e:
    print(f"\n Training failed with error: {str(e)}")
    raise

```

Epoch 1/3

163/163 0s 8s/step -

accuracy: 0.7917 - loss: 0.4362

Epoch 1: val_accuracy improved from -inf to 0.50000, saving model to

best_hybrid_cnn_transformer_1755109384.keras

163/163 1227s 8s/step -

accuracy: 0.7918 - loss: 0.4361 - val_accuracy: 0.5000 - val_loss: 2.2709 -

learning_rate: 0.0010

Epoch 2/3

163/163 0s 6s/step -

accuracy: 0.8457 - loss: 0.3389

Epoch 2: val_accuracy did not improve from 0.50000

163/163 943s 6s/step -

accuracy: 0.8457 - loss: 0.3390 - val_accuracy: 0.5000 - val_loss: 2.4211 -

learning_rate: 0.0010

Epoch 3/3

163/163 0s 6s/step -

accuracy: 0.8556 - loss: 0.3317

Epoch 3: val_accuracy did not improve from 0.50000
163/163 939s 6s/step -
accuracy: 0.8556 - loss: 0.3317 - val_accuracy: 0.5000 - val_loss: 2.3358 -
learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 1.

```
=====
TRAINING COMPLETED SUCCESSFULLY!
=====
```

```
Total training time: 51 minutes 49.9 seconds
Total epochs completed: 3
Best validation accuracy: 0.5000
Final training loss: 0.3383
Final validation loss: 2.3358
```

```
[26]: # 6. Training History Visualization
      # Plot training metrics to understand model performance

      import matplotlib.pyplot as plt

      def plot_training_history(history):
          """Plot training and validation metrics"""

          fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

          # Plot accuracy
          ax1.plot(history.history['accuracy'], label='Training Accuracy',
                  color='blue')
          ax1.plot(history.history['val_accuracy'], label='Validation Accuracy',
                  color='red')
          ax1.set_title('Model Accuracy')
          ax1.set_xlabel('Epoch')
          ax1.set_ylabel('Accuracy')
          ax1.legend()
          ax1.grid(True)

          # Plot loss
          ax2.plot(history.history['loss'], label='Training Loss', color='blue')
          ax2.plot(history.history['val_loss'], label='Validation Loss', color='red')
          ax2.set_title('Model Loss')
          ax2.set_xlabel('Epoch')
          ax2.set_ylabel('Loss')
          ax2.legend()
          ax2.grid(True)

          plt.tight_layout()
          plt.show()
```

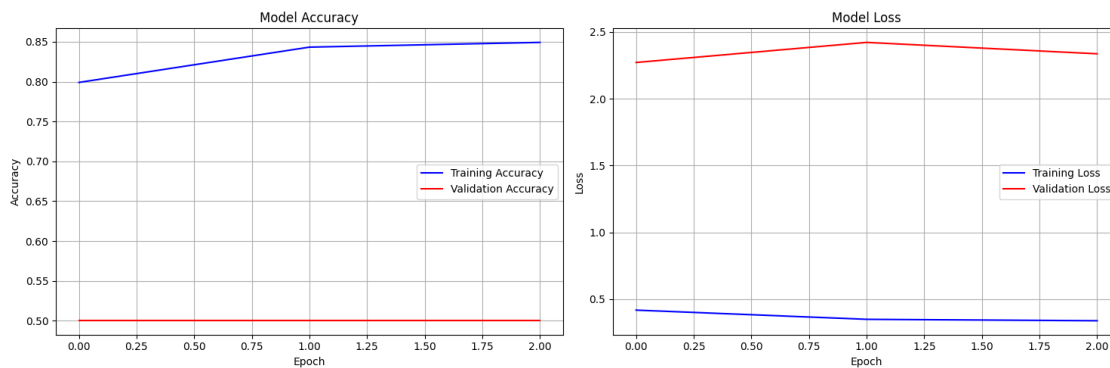
```

# Print final metrics
final_train_acc = history.history['accuracy'][-1]
final_val_acc = history.history['val_accuracy'][-1]
final_train_loss = history.history['loss'][-1]
final_val_loss = history.history['val_loss'][-1]

print(f" Final Training Metrics:")
print(f" Training Accuracy: {final_train_acc:.4f}")
print(f" Validation Accuracy: {final_val_acc:.4f}")
print(f" Training Loss: {final_train_loss:.4f}")
print(f" Validation Loss: {final_val_loss:.4f}")

# Call after training completes
plot_training_history(history)

```



```

Final Training Metrics:
Training Accuracy: 0.8493
Validation Accuracy: 0.5000
Training Loss: 0.3383
Validation Loss: 2.3358

```

5 6. Test the model

```

[ ]: # 7. Model Evaluation on Test Set
# Comprehensive evaluation using test data

import time
from sklearn.metrics import classification_report, confusion_matrix, \
    roc_auc_score, roc_curve

print(" Evaluating model on test set...")
start_time = time.time()

```



```

# Get predictions
test_generator.reset() # Reset generator
y_pred_prob = model.predict(test_generator, verbose=1)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = test_generator.classes

# Basic metrics
test_loss, test_accuracy = model.evaluate(test_generator, verbose=0)
end_time = time.time()

print(f"\n Test Set Results:")
print(f"    Test Accuracy: {test_accuracy:.4f}")
print(f"    Test Loss: {test_loss:.4f}")
print(f"    Evaluation Time: {end_time - start_time:.2f} seconds")
print(f"    Total Test Samples: {len(y_true)}")

```

Evaluating model on test set...
624/624 34s 54ms/step

Test Set Results:
 Test Accuracy: 0.6250
 Test Loss: 1.7059
 Evaluation Time: 69.89 seconds
 Total Test Samples: 624

6 7. Report the result

```

[28]: # 8. Detailed Performance Metrics
# Calculate precision, recall, F1-score, AUC

from sklearn.metrics import precision_recall_fscore_support

# Calculate metrics manually (since we removed them from compilation)
precision, recall, f1, support = precision_recall_fscore_support(y_true,
    ↪ y_pred, average=None)
avg_precision, avg_recall, avg_f1, _ = precision_recall_fscore_support(y_true,
    ↪ y_pred, average='weighted')

print(" Detailed Classification Metrics:")
print("\nPer-Class Metrics:")
for i, class_name in enumerate(['NORMAL', 'PNEUMONIA']):
    print(f"    {class_name}:")
    print(f"        Precision: {precision[i]:.4f}")
    print(f"        Recall: {recall[i]:.4f}")
    print(f"        F1-Score: {f1[i]:.4f}")
    print(f"        Support: {support[i]}")

```

```

print(f"\nWeighted Averages:")
print(f"    Precision: {avg_precision:.4f}")
print(f"    Recall: {avg_recall:.4f}")
print(f"    F1-Score: {avg_f1:.4f}")

# AUC Score
auc_score = roc_auc_score(y_true, y_pred_prob[:, 1])
print(f"    AUC Score: {auc_score:.4f}")

```

Detailed Classification Metrics:

Per-Class Metrics:

NORMAL:

Precision: 0.0000
Recall: 0.0000
F1-Score: 0.0000
Support: 234

PNEUMONIA:

Precision: 0.6250
Recall: 1.0000
F1-Score: 0.7692
Support: 390

Weighted Averages:

Precision: 0.3906
Recall: 0.6250
F1-Score: 0.4808
AUC Score: 0.3242

[29]: # 9. Confusion Matrix and ROC Curve Visualization

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['NORMAL', 'PNEUMONIA'],
            yticklabels=['NORMAL', 'PNEUMONIA'], ax=ax1)
ax1.set_title('Confusion Matrix')
ax1.set_xlabel('Predicted Label')
ax1.set_ylabel('True Label')

# ROC Curve
fpr, tpr, _ = roc_curve(y_true, y_pred_prob[:, 1])
ax2.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {auc_score:.2f})')

```

```

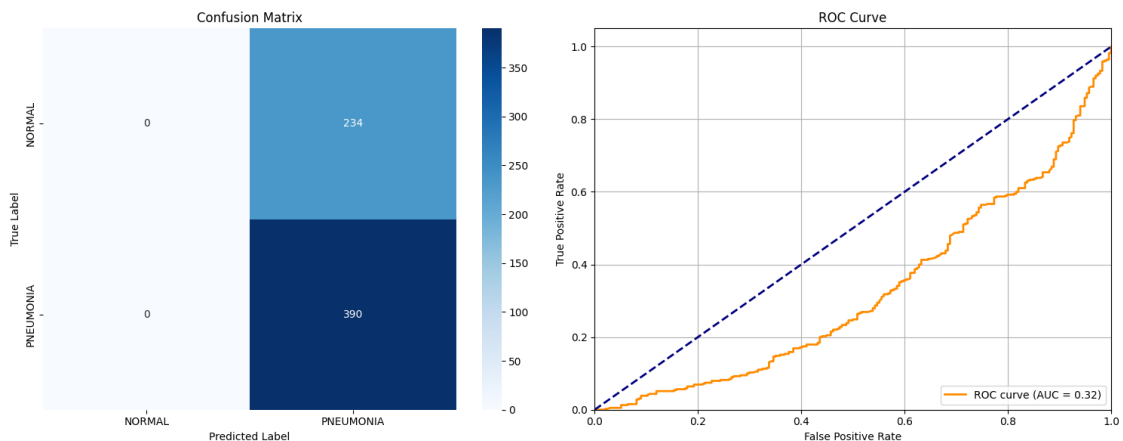
ax2.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
ax2.set_xlim([0.0, 1.0])
ax2.set_ylim([0.0, 1.05])
ax2.set_xlabel('False Positive Rate')
ax2.set_ylabel('True Positive Rate')
ax2.set_title('ROC Curve')
ax2.legend(loc="lower right")
ax2.grid(True)

plt.tight_layout()
plt.show()

# Additional metrics from confusion matrix
tn, fp, fn, tp = cm.ravel()
sensitivity = tp / (tp + fn) # Recall for PNEUMONIA
specificity = tn / (tn + fp) # Recall for NORMAL

print(f" Clinical Metrics:")
print(f" Sensitivity (True Positive Rate): {sensitivity:.4f}")
print(f" Specificity (True Negative Rate): {specificity:.4f}")
print(f" False Positive Rate: {fp/(fp+tn):.4f}")
print(f" False Negative Rate: {fn/(fn+tp):.4f}")

```



```

Clinical Metrics:
Sensitivity (True Positive Rate): 1.0000
Specificity (True Negative Rate): 0.0000
False Positive Rate: 1.0000
False Negative Rate: 0.0000

```

```

[32]: # 13. Final Report Summary
      # Generate comprehensive performance report

```

```

print("="*60)
print("  FINAL MODEL PERFORMANCE REPORT")
print("="*60)
print(f"Model: Hybrid CNN-Transformer for Pneumonia Detection")
print(f"Dataset: Chest X-ray Pneumonia Dataset")
print(f"Training Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print("="*60)
print(f"  PERFORMANCE METRICS:")
print(f"      • Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")
print(f"      • Precision (Weighted): {avg_precision:.4f}")
print(f"      • Recall (Weighted): {avg_recall:.4f}")
print(f"      • F1-Score (Weighted): {avg_f1:.4f}")
print(f"      • AUC Score: {auc_score:.4f}")
print(f"      • Sensitivity: {sensitivity:.4f}")
print(f"      • Specificity: {specificity:.4f}")
print("="*60)
print(f"  MODEL ARCHITECTURE:")
print(f"      • Total Parameters: {model.count_params():,}")
print(f"      • CNN Layers: 6 Convolutional + 3 MaxPooling")
print(f"      • Transformer Layers: 6")
print(f"      • Input Size: {IMG_SIZE}")
print(f"      • Batch Size: {BATCH_SIZE}")
print("="*60)

```

```

=====
FINAL MODEL PERFORMANCE REPORT
=====
Model: Hybrid CNN-Transformer for Pneumonia Detection
Dataset: Chest X-ray Pneumonia Dataset
Training Date: 2025-08-14 01:01:02
-----
PERFORMANCE METRICS:
    • Test Accuracy: 0.6250 (62.50%)
    • Precision (Weighted): 0.3906
    • Recall (Weighted): 0.6250
    • F1-Score (Weighted): 0.4808
    • AUC Score: 0.3242
    • Sensitivity: 1.0000
    • Specificity: 0.0000
-----
MODEL ARCHITECTURE:
    • Total Parameters: 895,458
    • CNN Layers: 6 Convolutional + 3 MaxPooling
    • Transformer Layers: 6
    • Input Size: (224, 224)
    • Batch Size: 32
-----

```